

University of Agriculture, Fsd

Lab Submission

Software Configuration Management - Lab

(SE 503)

Name : Muhammad Ahmad

Ag-no : 2022-ag-8042

Course : Nabeela Ashraf
Instructor

Semester : Fall 2024



Lab # 1

Introduction to GitHub for Configuration Management

Task:

1. Sign Up a GitHub account

Already have an account? [Sign in →](#)

Sign up to GitHub

Email*

mahmadtayyab@gmail.com

Password*

.....

Password should be at least 15 characters OR at least 8 characters including a number and a lowercase letter.

Username*

Ahmad-tayyab-11|

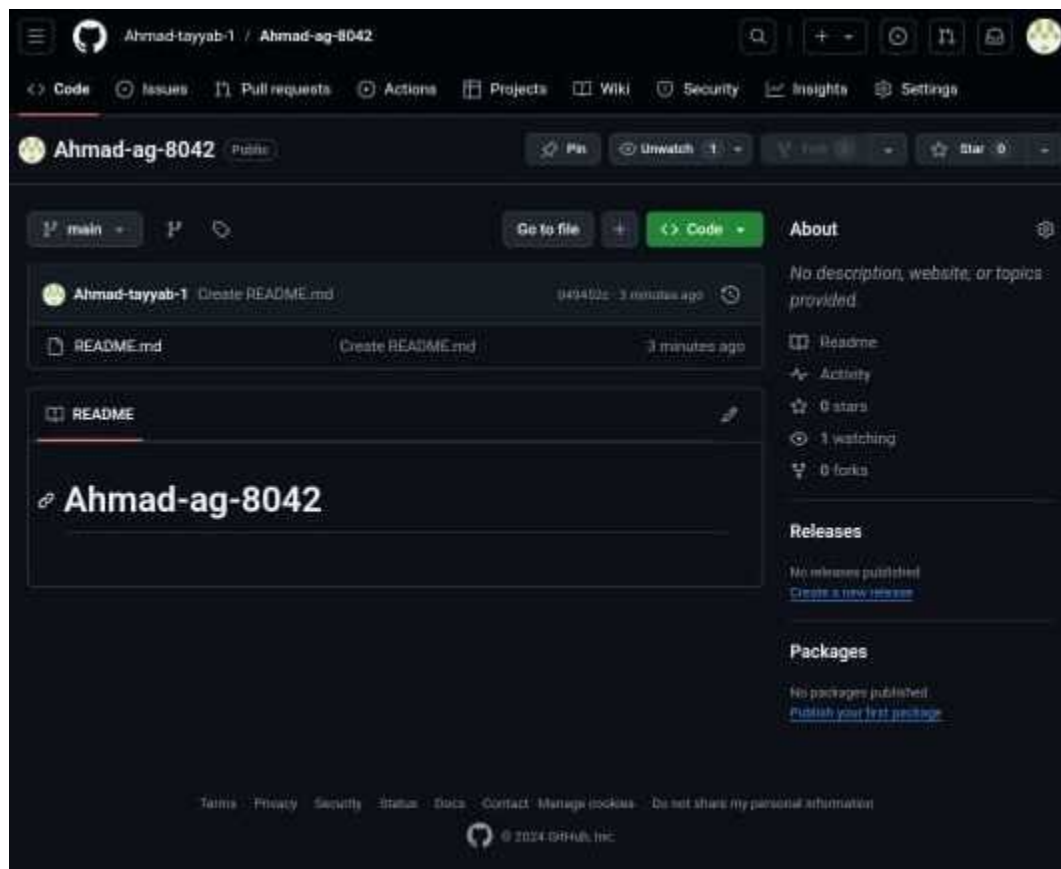
Ahmad-tayyab-11 is available.

Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.

By creating an account, you agree to the [Terms of Service](#). For more information about GitHub's privacy practices, see the [GitHub Privacy Statement](#). We'll occasionally send you account-related emails.

Continue >

2. Create a repository named **Ahmad-ag-8042**

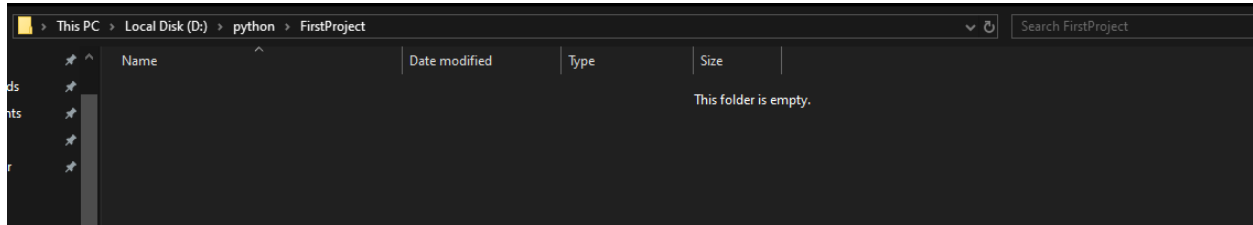


Lab # 2

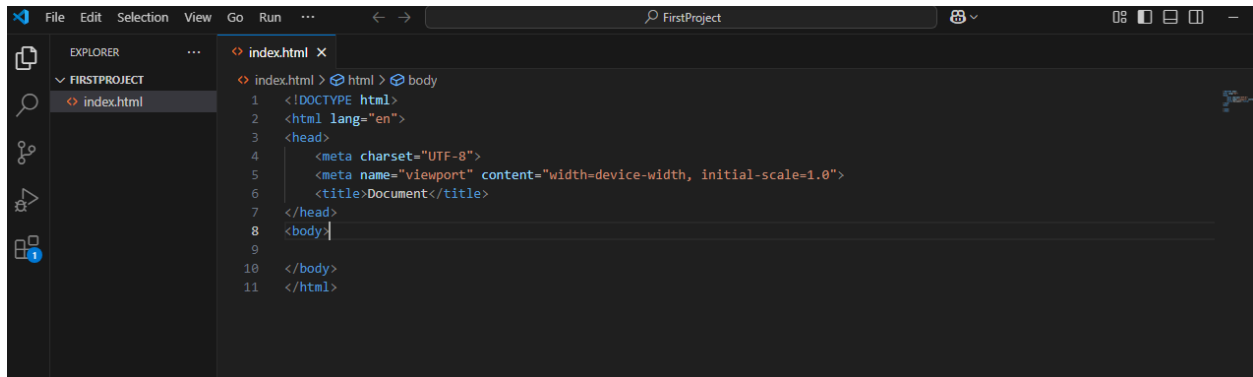
Setting Up a Development Environment and Working with Git

Task 1: Creating and Managing Commits.

- Create a New Local Repository:



- Make changes and stage files



- Commit the changes

Use the **git add cmd** to add files to commit

Use the **git commit -m "initial message"** to commit the files

```
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">

PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE powershell
PS D:\python\FirstProject> git init -b main
Initialized empty Git repository in D:/python/FirstProject/.git/
PS D:\python\FirstProject> git add .
PS D:\python\FirstProject> git commit -m "initial commit For First Project"
[main (root-commit) 27a3f2f] initial commit For First Project
1 file changed, 11 insertions(+)
create mode 100644 index.html
PS D:\python\FirstProject>
```

- Push the changes to GitHub

Use the **git push -u origin main** to Push the changes to **GitHub**

```
PS D:\python\FirstProject> git branch -M main
PS D:\python\FirstProject> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 379 bytes | 189.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Ahmad-tayyab-1/FirstProject.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS D:\python\FirstProject>
```

- View commit history

Using the **git log cmd** to check commit history to verify

```
PS D:\python\FirstProject> git log
commit 27a3f2f7e4e02029c5d941054c732ba436815776 (HEAD -> main, origin/main)
Author: Ahmad-tayyab-1 <mahmadtayyab45@gmail.com>
Date: Thu Jan 2 13:25:24 2025 +0500

    initial commit For First Project
PS D:\python\FirstProject>
```

Task 2:

1. Create a New Branch

Use **git branch <new-branch-name>**

Navigate to new branch use **git checkout -b <branch-name>**

```
PS D:\python\FirstProject> git checkout -b Second-branch
Switched to a new branch 'Second-branch'
PS D:\python\FirstProject> git add index.html
```

2. Make Changes in the New Branch

Use **git add** and **git commit -m** to make changes in the new branch

```
no changes added to commit (use "git add" and/or "git commit -a")
PS D:\python\FirstProject> git add index.html
PS D:\python\FirstProject> git commit -m "adding files to new branch"
[Second-branch c122572] adding files to new branch
1 file changed, 1 insertion(+), 1 deletion(-)
PS D:\python\FirstProject>
```

3. Switch Back to the Master Branch

Use **git checkout main** to switch back to main branch

```
PS D:\python\FirstProject> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS D:\python\FirstProject>
```

4. Merge the Feature Branch into Master

Use **git merge <branch-name>** to merge the new branch to main branch

```
PS D:\python\FirstProject> git merge Second-branch
Updating 27a3f2f..c122572
Fast-forward
 index.html | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
PS D:\python\FirstProject>
```

5. Push the Merged Changes to GitHub

Use **git push -u origin main** to push the changes to github

```
Fast-forward
 index.html | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
PS D:\python\FirstProject> git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 311 bytes | 311.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Ahmad-tayyab-1/FirstProject.git
 27a3f2f..c122572 main -> main
branch 'main' set up to track 'origin/main'.
PS D:\python\FirstProject>
```

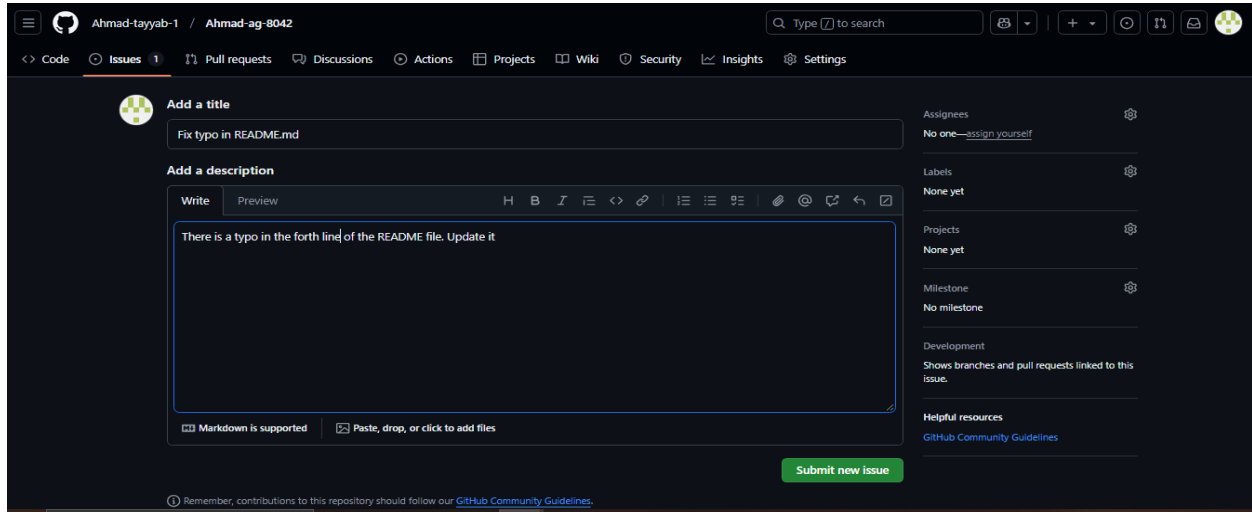
Lab # 3

Advanced GitHub Features

Task 1: Creating and Managing Issues

1. Create an Issue:

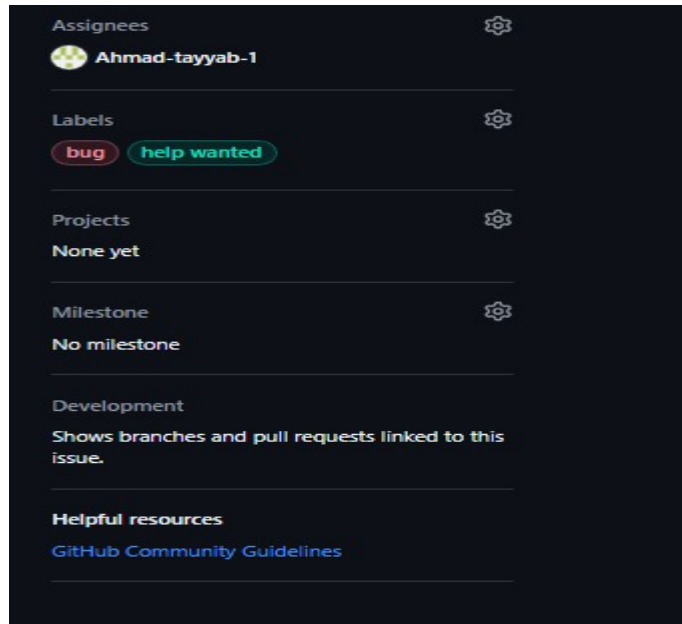
- Open a repository on GitHub.
- Navigate to the Issues tab and click on New Issue.
- Fill in the issue title and description to specify a task or bug.
- Click Submit new issue.



The screenshot shows the GitHub 'New Issue' interface. At the top, the repository name 'Ahmad-tayyab-1 / Ahmad-ag-8042' is displayed. Below the navigation bar, the 'Issues' tab is selected. The form is titled 'Add a title' and contains the text 'Fix typo in README.md'. Below this is the 'Add a description' section, which has a 'Write' tab and a 'Preview' tab. The description text is 'There is a typo in the forth line of the README file. Update it'. To the right of the form, there are sections for 'Assignees' (No one—assign yourself), 'Labels' (None yet), 'Projects' (None yet), and 'Milestone' (No milestone). At the bottom right, there is a green 'Submit new issue' button. A footer note states: 'Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).'

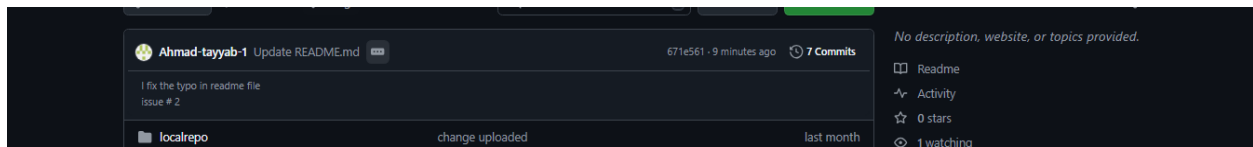
2. Assign the Issue and labels:

- Assign the issue to yourself or another team member.
- Add a due date if necessary.
- Apply labels like bug, enhancement, or help wanted to categorize the issue.
- Labels help in filtering and prioritizing tasks.



3. Close the Issue:

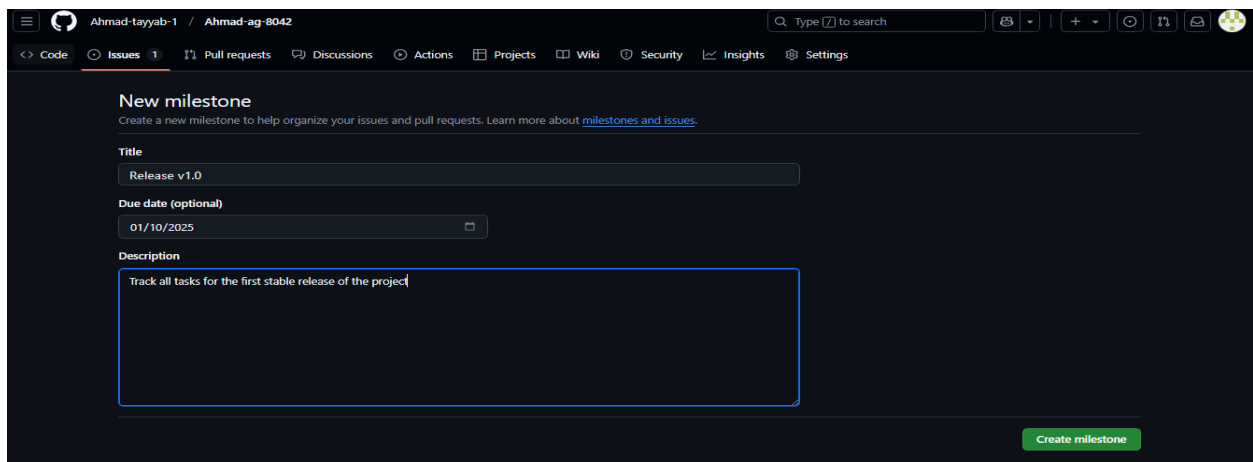
- After completing the task, reference the issue in a commit by including Fixes #<issue-number> in the commit message.
- Example: I fixes issue #1.



Task 2: Planning with Milestones

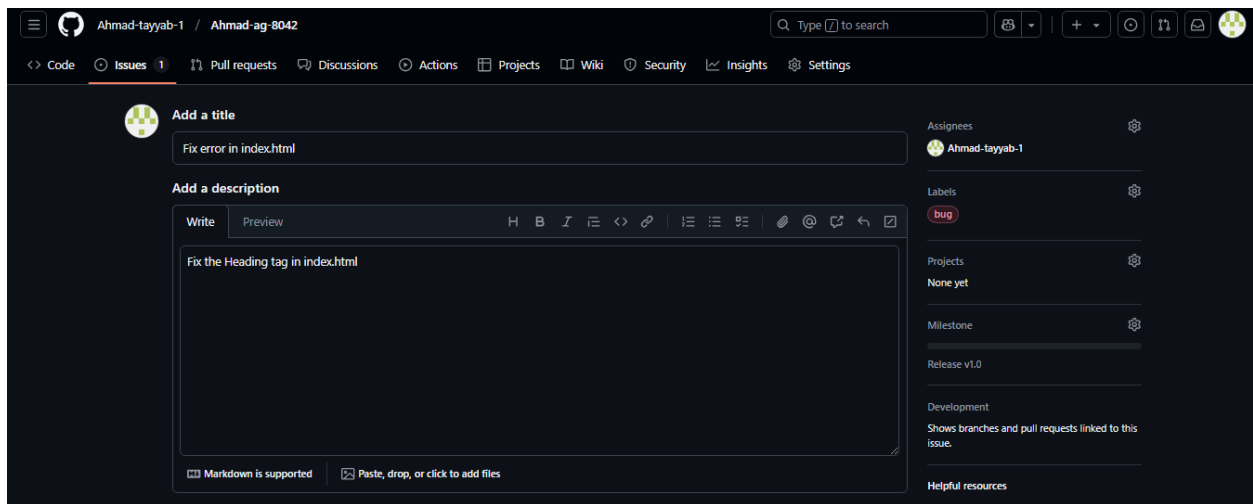
1. Create a Milestone:

- Set a title, description, and due date for the milestone.

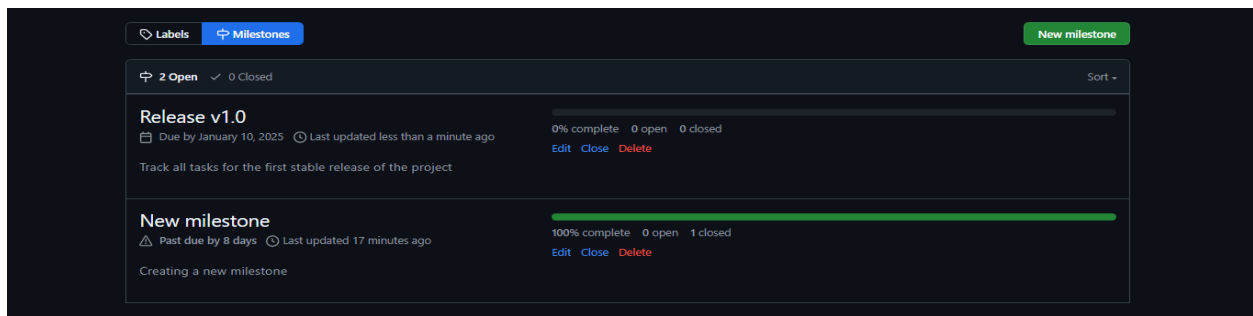


2. Add Issues to Milestone:

- Open an issue and assign it to the milestone.
- Use multiple issues to plan tasks related to the milestone.

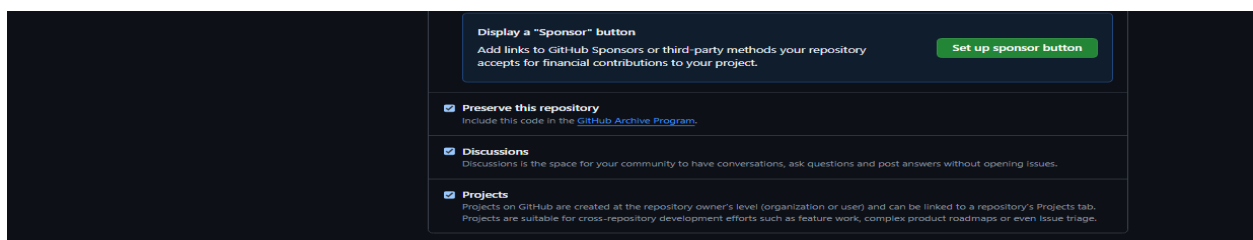


3. Track Milestone Progress:



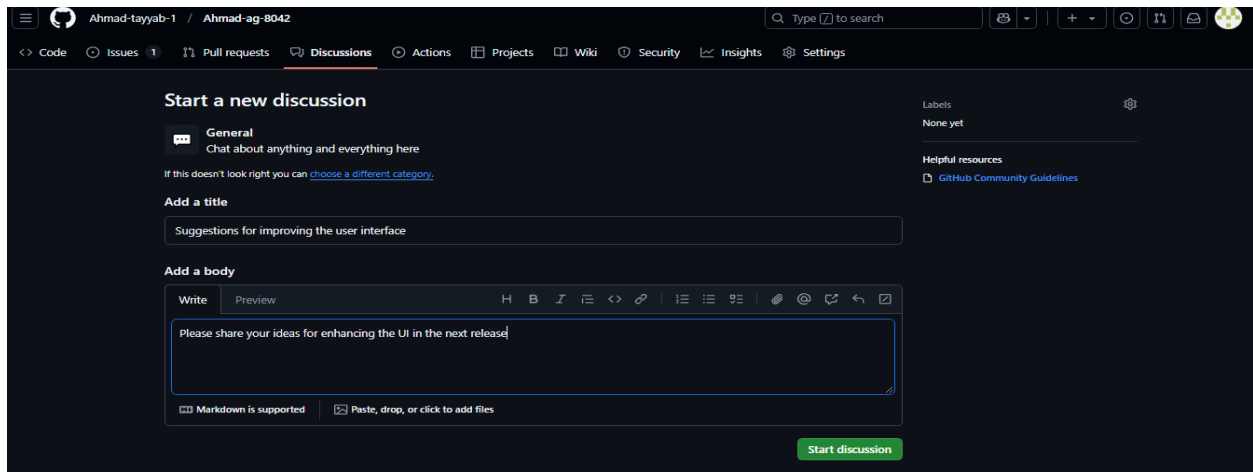
Task 3: Collaborating with GitHub Discussions

1. Enable Discussions (if not already enabled) in repository setting:

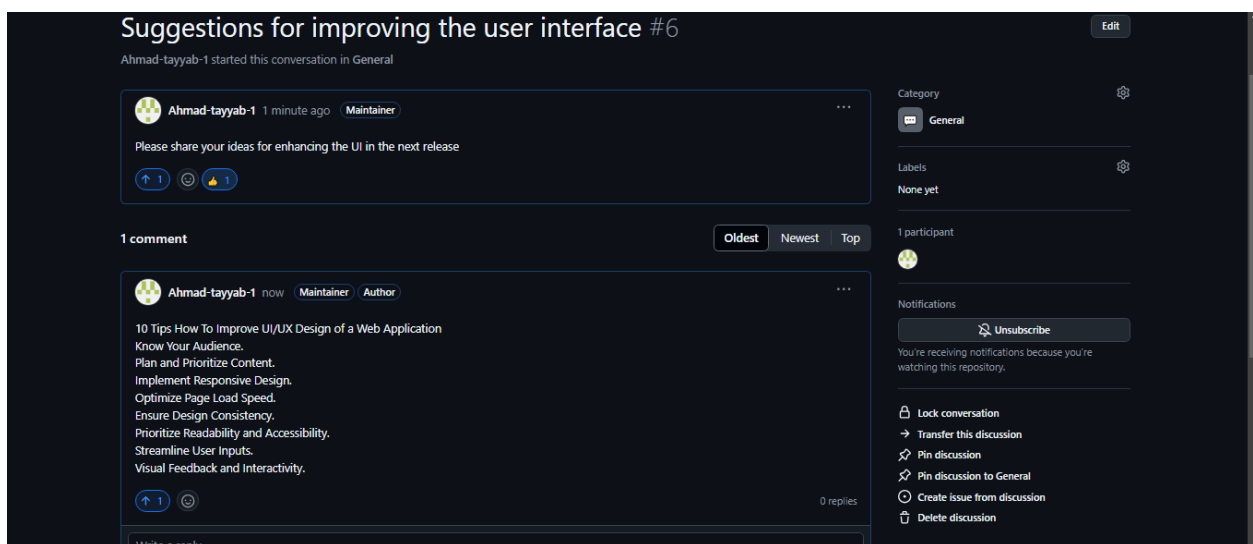


2. Start a Discussion:

- Navigate to the Discussions tab and click New Discussion.
- Select the discussion category (e.g., General, Q&A).



3. Participate in Discussions:
 - Reply to existing discussion topics.

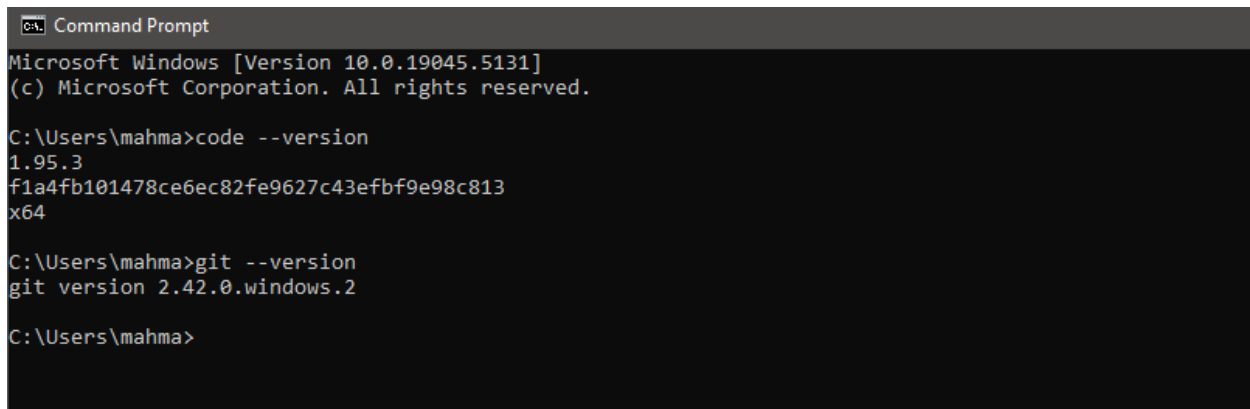


Lab # 4

Setting Up a Development Environment and Working with Git

Task 1: Download and Install VS Code and Git

- Downloaded Visual Studio Code from the official website.
- Downloaded Git from the official Git website.
- verified the installation using commands like **git --version** and **code --version**



```
Command Prompt
Microsoft Windows [Version 10.0.19045.5131]
(c) Microsoft Corporation. All rights reserved.

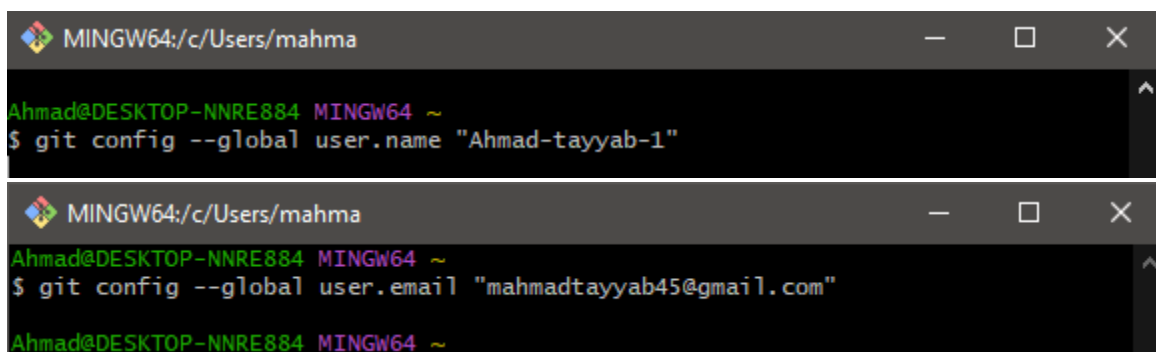
C:\Users\mahma>code --version
1.95.3
f1a4fb101478ce6ec82fe9627c43efbf9e98c813
x64

C:\Users\mahma>git --version
git version 2.42.0.windows.2

C:\Users\mahma>
```

Task 2: Configure Git

- Configured Git with my name and email using the commands:
- **git config --global user.name "Your Name"**
- **git config --global user.email "Your Email"**



```
MINGW64:/c/Users/mahma
Ahmad@DESKTOP-NNRE884 MINGW64 ~
$ git config --global user.name "Ahmad-tayyab-1"

MINGW64:/c/Users/mahma
Ahmad@DESKTOP-NNRE884 MINGW64 ~
$ git config --global user.email "mahmadtayyab45@gmail.com"
Ahmad@DESKTOP-NNRE884 MINGW64 ~
```

- Verified the configuration using **git config --list**.

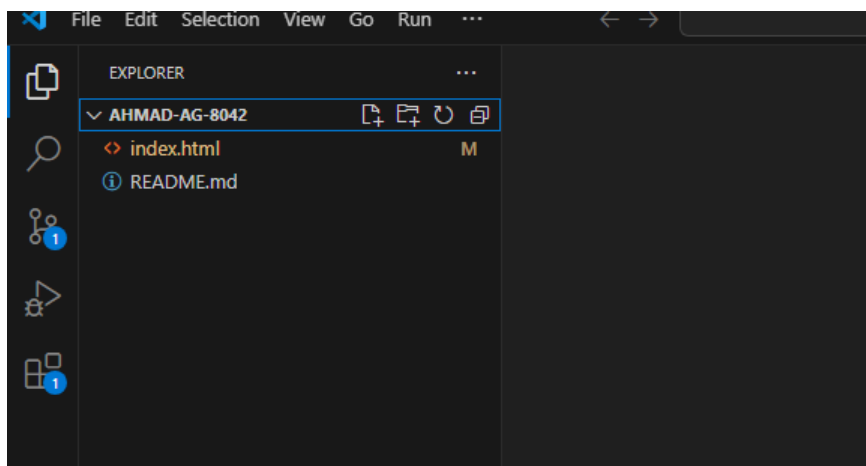
```
MINGW64:/c/Users/mahma
Ahmad@DESKTOP-NNRE884 MINGW64 ~
$ git config --global user.email "mahmadtayyab45@gmail.com"

Ahmad@DESKTOP-NNRE884 MINGW64 ~
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Ahmad-tayyab-1
user.email=mahmadtayyab45@gmail.com

Ahmad@DESKTOP-NNRE884 MINGW64 ~
$ |
```

Task 4: Create a Folder and Open in VS Code:

- Created a folder named **Ahmad-AG-8042** and opened it in VS Code.



Task 5: Clone a Repository from GitHub:

- Open Integrated Terminal in VS Code:
- Cloned a GitHub repository by using the **git clone <repo link>**

```
PS D:\python\SE-503\Ahmad-ag-8042> git clone https://github.com/Ahmad-tayyab-1/Ahmad-ag-8042.git
Cloning into 'Ahmad-ag-8042'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.
PS D:\python\SE-503\Ahmad-ag-8042> █
```

Task 6: Open and Modify Files:

- Opened a file from the cloned repository, made some changes, and saved them.

Task 7: Check Git Status:

- Use the **git status** command to check the status of files.
- Observed the file states such as untracked, modified, staged, or deleted.

```
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
PS D:\python\SE-503\Ahmad-ag-8042> █
```

Lab # 5

Version Control with Git: Managing Changes, Uploading Projects to GitHub, and Working with Branches

Task 1: Stage and Commit Changes

1. Open the terminal in the cloned repository folder (from the previous lab).

```
PS D:\python\SE-503\Ahmad-ag-8042>
* History restored
PS D:\python\SE-503\Ahmad-ag-8042>
```

2. View the status of your repository:

Use **git status** cmd to check the status

```
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
PS D:\python\SE-503\Ahmad-ag-8042>
```

3. Stage the modified or new files:

- **git add <index.html>**

To stage all changes at once: **use git add .**

4. Commit the staged changes with a descriptive message:

- **git commit -m "Updated the index .html"**

```
PS D:\python\SE-503\Ahmad-ag-8042> git commit -m "updated the index .html "
• [main 556f4c9] updated the index .html
  1 file changed, 1 insertion(+)
○ PS D:\python\SE-503\Ahmad-ag-8042>
```

Task 2: Push Changes to GitHub

1. If someone has updated the repository on GitHub, your local copy may be outdated, and Git prevents you from pushing to avoid overwriting others. Execute the following commands to update the local repo:

git fetch origin

git pull origin <branch_name>

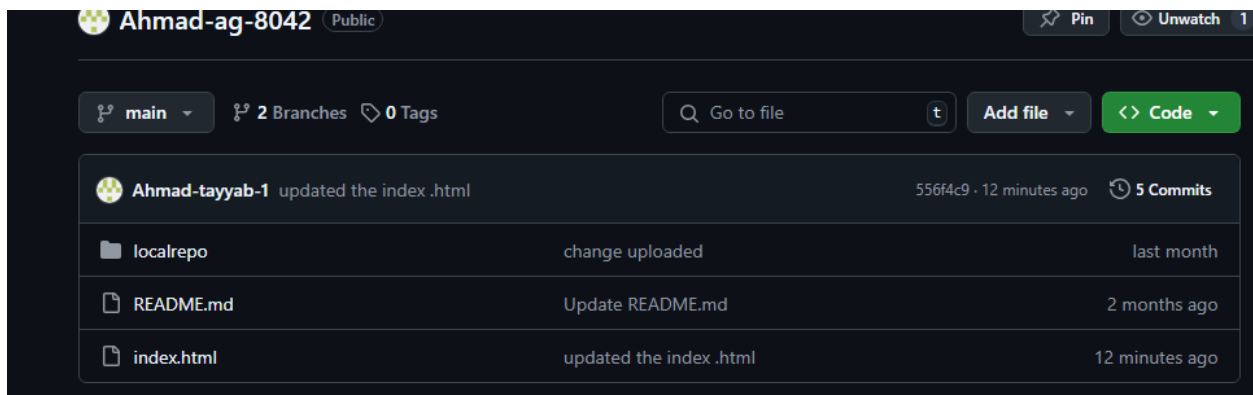
```
PROBLEMS  OUTPUT  TERMINAL  PORTS  DEBUG CONSOLE
PS D:\python\SE-503\Ahmad-ag-8042> git fetch
• PS D:\python\SE-503\Ahmad-ag-8042> git pull origin main
From https://github.com/Ahmad-tayyab-1/Ahmad-ag-8042
* branch      main      -> FETCH_HEAD
Already up to date.
○ PS D:\python\SE-503\Ahmad-ag-8042>
```

2. Push your committed changes to the GitHub repository:

git push origin <branch_name>

```
Already up to date.
PS D:\python\SE-503\Ahmad-ag-8042> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 383 bytes | 383.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Ahmad-tayyab-1/Ahmad-ag-8042.git
 30bba1e..556f4c9  main -> main
PS D:\python\SE-503\Ahmad-ag-8042> |
```

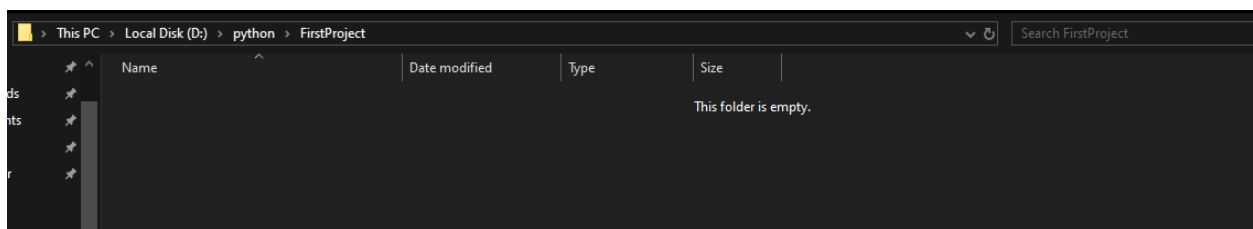
3. Verify the changes on the GitHub repository by visiting the repository page in your browser



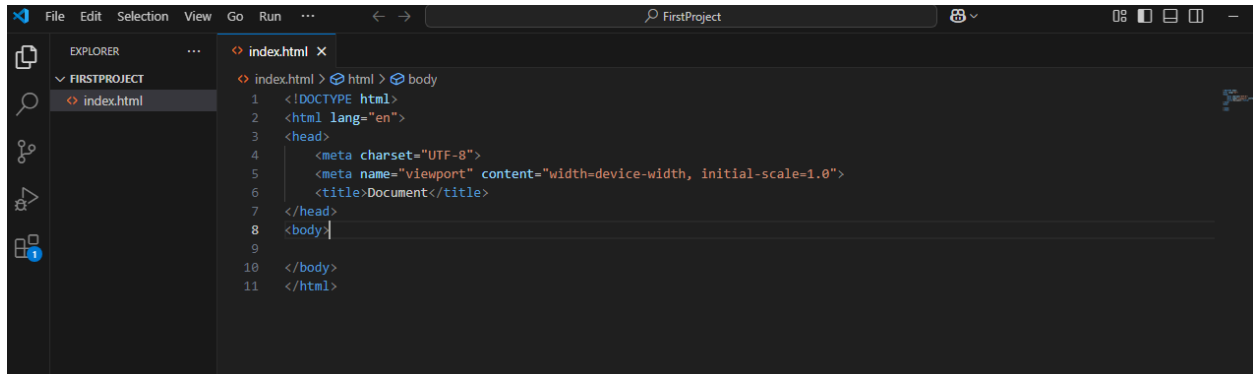
Task 3: Create a New Project on Your Desktop and Upload it to GitHub

Create a New Project Locally

1. Create a new folder on your desktop (e.g., FirstProject).

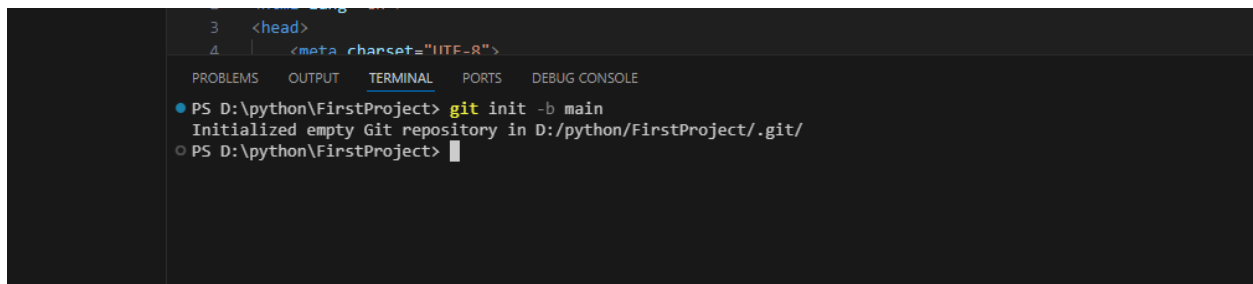


2. Inside the folder, create files for your project (e.g., .txt, .html, .py, etc.).



Step 2: Initialize Git in the Folder

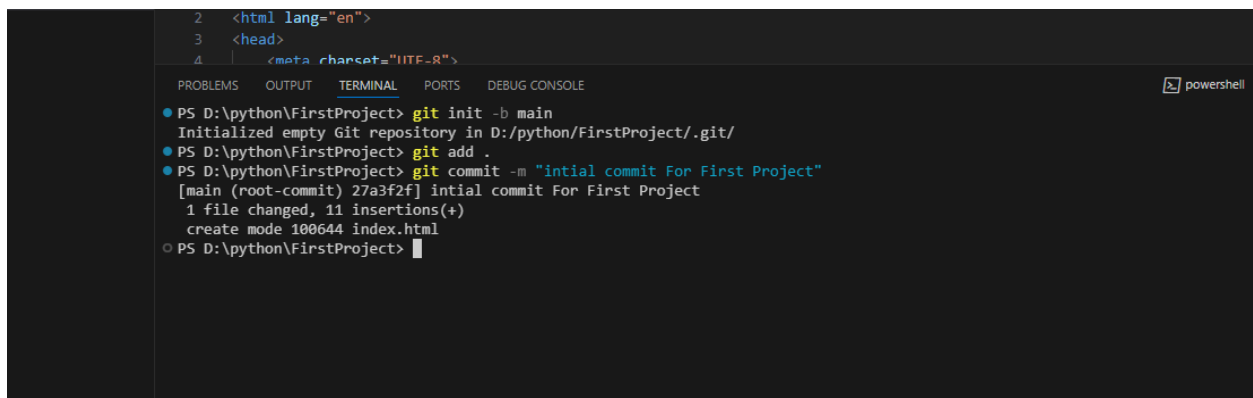
1. Initialize Git in this folder using the git init command



Step 3: Add and Commit Files

1. Add all files to the staging area using the **git add** .
2. Commit the files with a descriptive message:

`git commit -m "Initial commit for FirstProject"`



Step 4: Create a New GitHub Repository

1. Log in to [GitHub](#).
2. Click the **New** button to create a repository.
3. Provide a repository name (e.g., FirstProject) and description (optional).
4. Leave the repository **empty** (do not initialize with a README).
5. Click **Create Repository**.

Create a new repository
A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner * Ahmad-tayyab-1 Repository name * FirstProject
FirstProject is available.

Great repository names are short and memorable. Need inspiration? How about [supreme-umbrella](#)?

Description (optional)

☒ **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

☐ **Add a README file**
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Step 5: Link the Local Repository to GitHub

1. Copy the repository's URL (e.g., <https://github.com/username/FirstProject.git>).
2. Link your local repository to the GitHub repository:

git remote add origin <https://github.com/Ahmad-tayyab-1/FirstProject.git>

```
Create mode 100644 index.html
PS D:\python\FirstProject> git remote add origin https://github.com/Ahmad-tayyab-1/FirstProject.git
PS D:\python\FirstProject>
```

3. Verify either remote command worked or not

git remote -v

```
PS D:\python\FirstProject> git remote add origin https://github.com/Ahmad-tayyab-1/FirstProject.git
PS D:\python\FirstProject> git remote -v
origin https://github.com/Ahmad-tayyab-1/FirstProject.git (fetch)
origin https://github.com/Ahmad-tayyab-1/FirstProject.git (push)
PS D:\python\FirstProject>
```

4. Check the current branch using **git branch**

Rename the branch from master to main (the convention github follows)

Using branch -M main

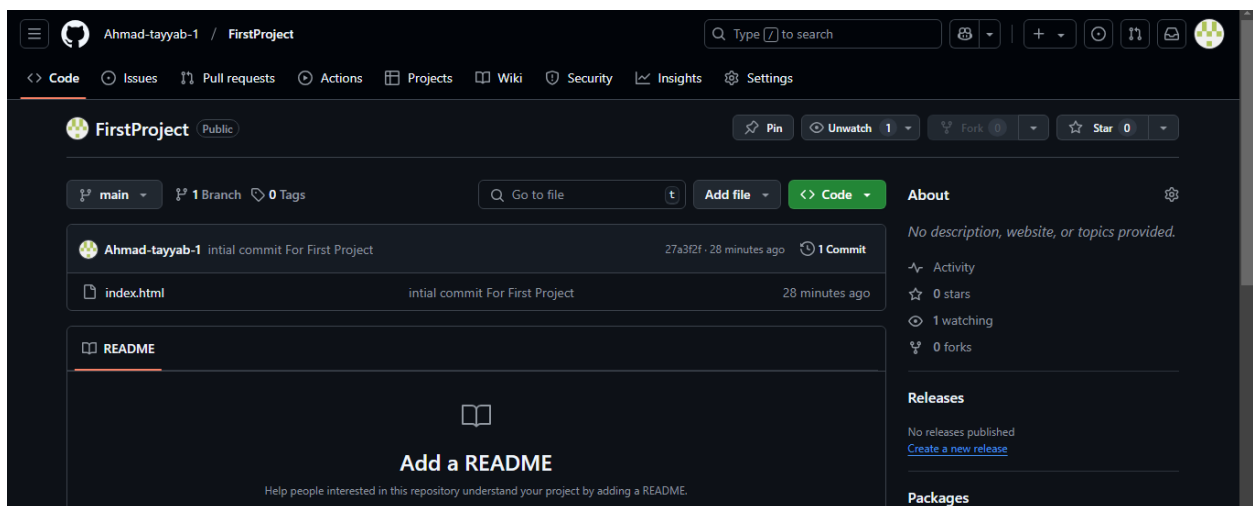
```
origin: https://github.com/Ahmad-tayyab-1/FirstProject.git (push)
• PS D:\python\FirstProject> git branch
* main
• PS D:\python\FirstProject> git branch -M main
○ PS D:\python\FirstProject>
```

6. Push the project to GitHub:

git push -u origin main

```
• PS D:\python\FirstProject> git branch -M main
• PS D:\python\FirstProject> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 379 bytes | 189.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Ahmad-tayyab-1/FirstProject.git
 * [new branch]    main -> main
branch 'main' set up to track 'origin/main'.
○ PS D:\python\FirstProject>
```

Verify the Upload Files on Github by opening the “FirstProject Repository” in your github account



Lab # 6

Working with Branches: Creating, Modifying, Merging, Simulating, and Resolving Conflicts

Task 1 :Resolving Merge Conflict

- Create a new branch “**feature1**”

Using **git branch feature1**

- Navigate to **feature1** branch

Using **git checkout feature1**

```
fatal: a branch named 'feature1' already exists
PS D:\python\FirstProject> git branch feature-1
PS D:\python\FirstProject> git checkout feature1
Switched to branch 'feature1'
PS D:\python\FirstProject> 
```

- Make changes in the **feature branch** and commit them

Using **git add index.html** and **git commit -m “feature1 edit”**

```
PS D:\python\FirstProject> git checkout feature1
Switched to branch 'feature1'
PS D:\python\FirstProject> git add index.html
1 file changed, 1 insertion(+)
```

- Navigate back to main branch and make changes in the same file as feature1 branch

Using **git checkout main**

- Merge the feature1 branch to main branch

```

1 file changed, 1 insertion(+)
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS D:\python\FirstProject> git add index.html
PS D:\python\FirstProject> git commit -m "feature1 edit"
[main 5b18108] feature1 edit
1 file changed, 1 insertion(+)
PS D:\python\FirstProject> git merge feature1
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html

```

- Resolve the merge conflict by manually editing the file of conflict

```

index.html | x
index.html > html > body > ?
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Document</title>
7 </head>
8 <body>
9 | <p>Hello</p>
10 <==== HEAD (Current Change)
11 <p>New World</p>
12 =====
13 <p>World</p>
14 >==== feature1 (Incoming Change)
15 </body>
16 </html>
Resolve in Merge Editor

PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE
PS D:\python\FirstProject> git commit -m "feature1 edit"
[feature1 01f14b7] feature1 edit
1 file changed, 1 insertion(+)
PS D:\python\FirstProject> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS D:\python\FirstProject> git add index.html
PS D:\python\FirstProject> git commit -m "feature1 edit"
[main 5b18108] feature1 edit
1 file changed, 1 insertion(+)
PS D:\python\FirstProject> git merge feature1
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
PS D:\python\FirstProject>

```

- After resolving the issue merge the branch

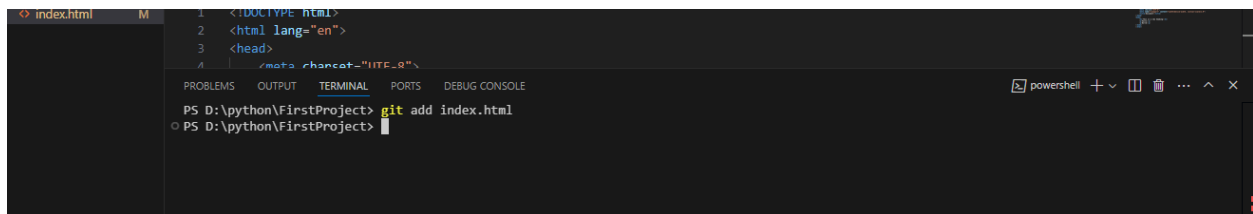
```

PS D:\python\FirstProject> git add index.html
PS D:\python\FirstProject> git commit -m "accept the feature branch change"
[main a05cd27] accept the feature branch change
PS D:\python\FirstProject> git merge feature1
Already up to date.

```

Task 6: Undo Mistake in Git

1. Modify a file and add it to the staging area:
 - Using **git add index.html**



The screenshot shows the Visual Studio Code editor with a file named `index.html` open. The file content is as follows:

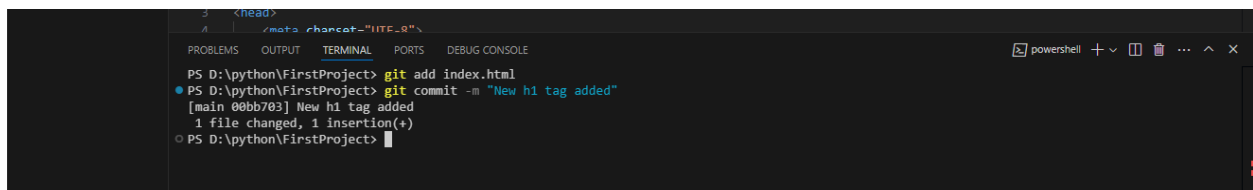
```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
```

The terminal window at the bottom shows the following commands and output:

```
PS D:\python\FirstProject> git add index.html
PS D:\python\FirstProject>
```

2. Commit the add:

- Using **git commit -m "New h1 tag added"**

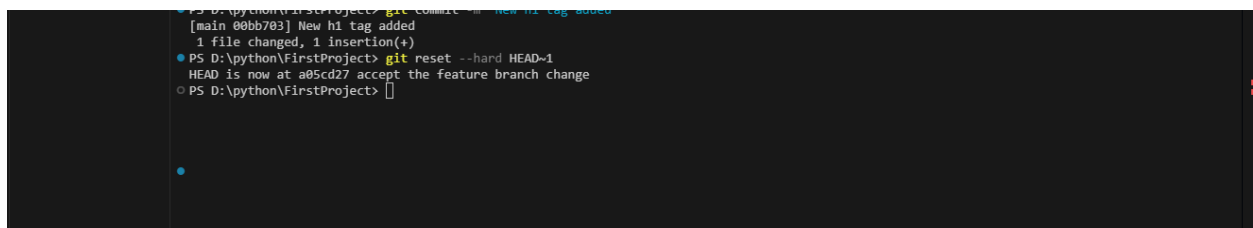


The screenshot shows the Visual Studio Code terminal window with the following output:

```
PS D:\python\FirstProject> git add index.html
PS D:\python\FirstProject> git commit -m "New h1 tag added"
[main 00bb703] New h1 tag added
1 file changed, 1 insertion(+)
PS D:\python\FirstProject>
```

3. Undo the last commit while keeping the changes:

- Using **git reset --hard HEAD~1**



The screenshot shows the Visual Studio Code terminal window with the following output:

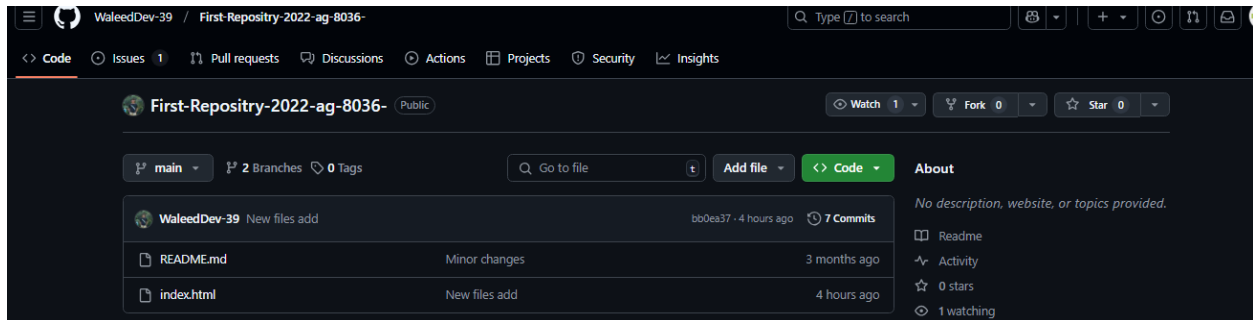
```
PS D:\python\FirstProject> git commit -m "New h1 tag added"
[main 00bb703] New h1 tag added
1 file changed, 1 insertion(+)
PS D:\python\FirstProject> git reset --hard HEAD~1
HEAD is now at a05cd27 accept the feature branch change
PS D:\python\FirstProject>
```

Lab # 7

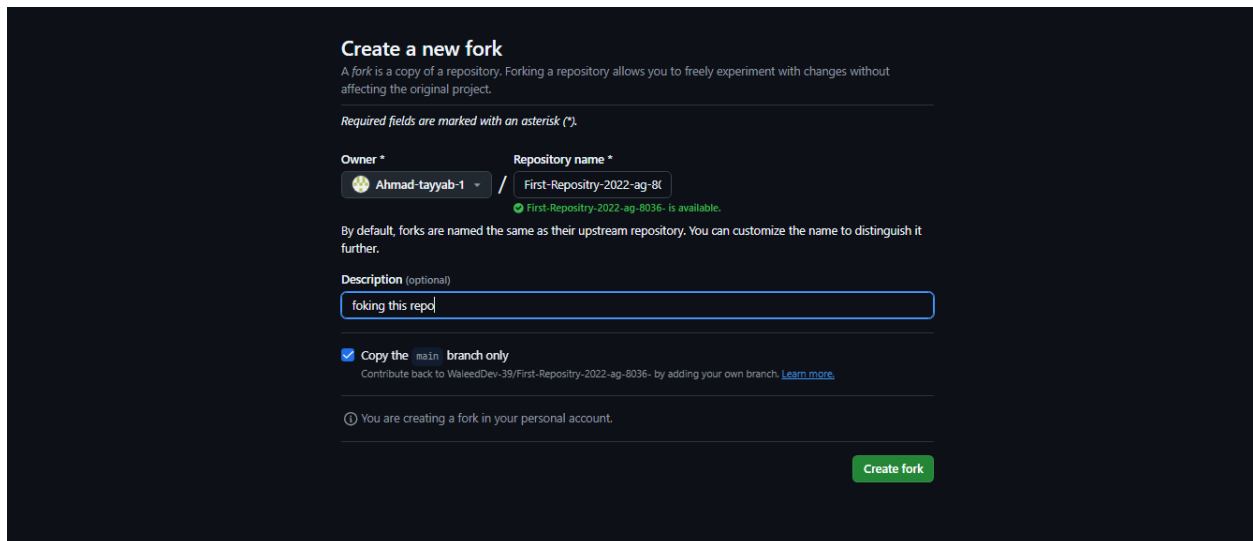
Advanced GitHub Features (Including Forking)

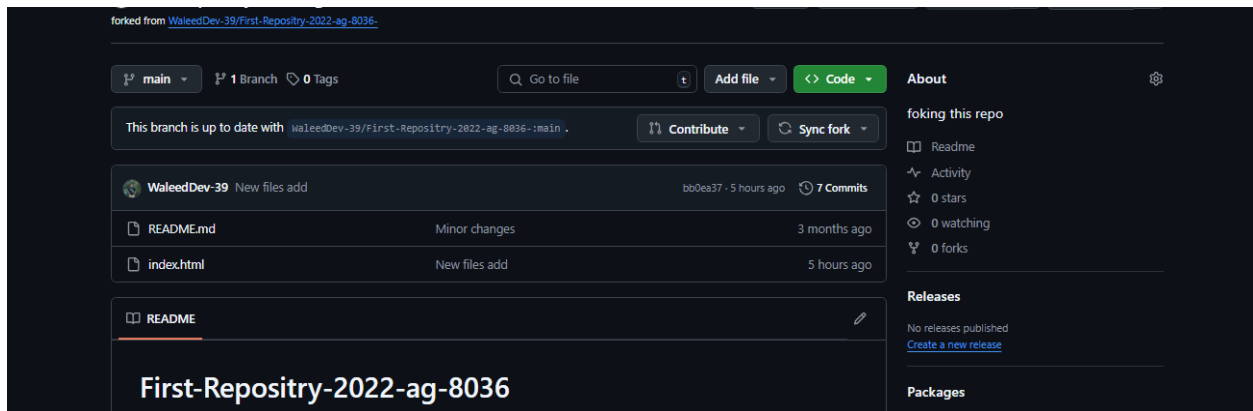
Task 1: Forking a Repository

1. Fork a Repository:
 - Navigate to any public repository on GitHub (e.g., an open-source project or a provided practice repository).



- Click the Fork button in the top-right corner to create your own copy of the repository.





2. Clone the Forked Repository Locally:
 - Clone your forked repository to your local machine:

<https://github.com/Ahmad-tayyab-1/First-Repository-2022-ag-8036-.git>

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.5247]
(c) Microsoft Corporation. All rights reserved.

D:\python\fork repo>git clone https://github.com/Ahmad-tayyab-1/First-Repository-2022-ag-8036-.git
Cloning into 'First-Repository-2022-ag-8036-'...
remote: Enumerating objects: 21, done.
remote: Counting objects: 100% (21/21), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 21 (delta 4), reused 10 (delta 3), pack-reused 0 (from 0)
Receiving objects: 100% (21/21), 4.88 KiB | 499.00 KiB/s, done.
Resolving deltas: 100% (4/4), done.

D:\python\fork repo>
```

3. Make Changes in Your Fork:
 - Edit files or add new features in the forked repository.

4. Commit and Push Changes:
 - Stage and commit your changes:

git add index.html

git commit -m "Added new tag in index.html"

```

PS D:\python\fork repo\First-Repository-2022-ag-8036-> git add index.html
PS D:\python\fork repo\First-Repository-2022-ag-8036-> git commit -m "added new html tag in index.html"
[main caa2861] added new html tag in index.html
1 file changed, 1 insertion(+), 4 deletions(-)

```

Push changes to your fork on GitHub:

git push -u origin main

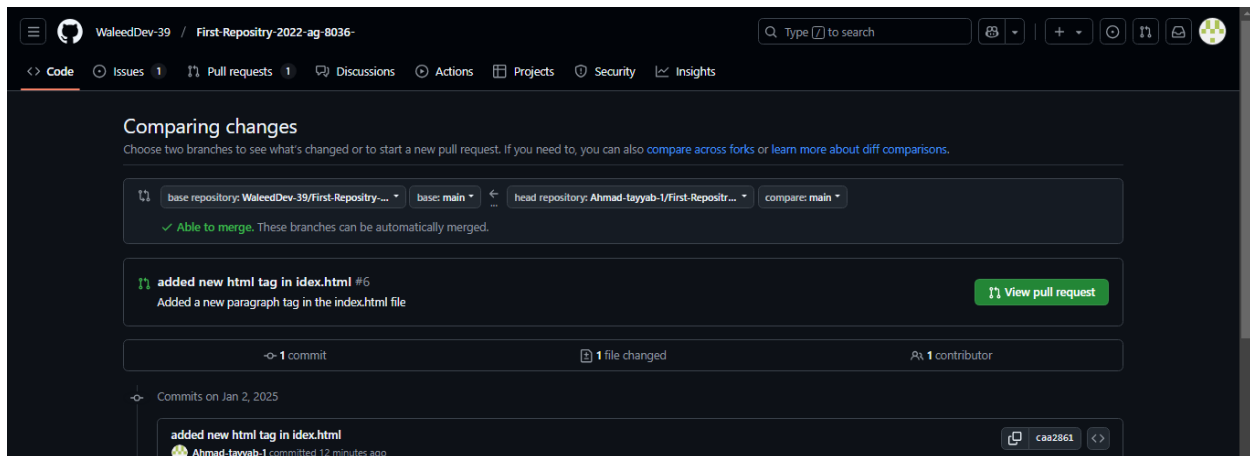
```

PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE
PS D:\python\fork repo\First-Repository-2022-ag-8036-> git add index.html
PS D:\python\fork repo\First-Repository-2022-ag-8036-> git commit -m "added new html tag in index.html"
[main caa2861] added new html tag in index.html
1 file changed, 1 insertion(+), 4 deletions(-)
PS D:\python\fork repo\First-Repository-2022-ag-8036-> git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 363 bytes | 363.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Ahmad-tayyab-1/First-Repository-2022-ag-8036-.git
bb0ea37..caa2861 main -> main
branch 'main' set up to track 'origin/main'.
PS D:\python\fork repo\First-Repository-2022-ag-8036->

```

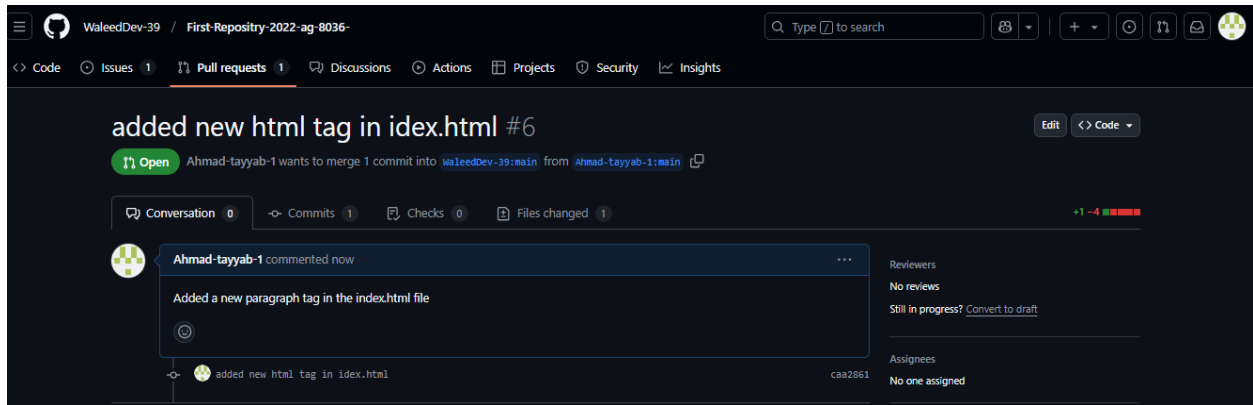
5. Propose Changes to the Original Repository (Pull Request):

- Navigate to your forked repository on GitHub.
- Click **Pull Requests > New Pull Request**.



Compare your changes with the original repository and submit the pull request.

- Add a **title** and **description** to explain your changes.



6. Track and Respond to Pull Request Reviews:

- Respond to comments or suggestions made by the original repository's maintainers.
- If changes are requested, update your branch and the pull request.

